Ozzie the Wizard

Project Anatomy — v1.0

A native macOS menu bar task manager built with Swift & SwiftUI. This document maps every file in the project, its purpose, key types, and how it connects to the rest of the system.

Platform	macOS 14.0+ (Sonoma)
Language	Swift 5.9
UI Framework	SwiftUI + AppKit
Build System	XcodeGen → Xcode
Storage	Local JSON (~/ Library/Application Support/Ozzie/)
Bundle ID	com.ozzie.wizard
Hotkeys	Cmd+Option+N / A / S

Table of Contents

1. Architecture Overview

Ozzie follows a **layered architecture** with clear separation of concerns:

Layer	Files	Responsibility
Entry	OzzieApp.swift	App bootstrap, creates AppDelegate
Coordination	AppDelegate.swift	Menu bar, popover, hotkey routing
Models	OzzieTask / SubTask / TaskContext	Data structures, computed properties
Storage	StorageManager.swift	JSON persistence, CRUD, search
Views	5 SwiftUI views	All UI: list, detail, creation, picker
Services	4 service singletons	Hotkeys, screenshots, OCR, notifications
Resources	WizardIcon.swift + config files	Pixel art icon, plist, entitlements

Key design decisions:

- **No dock icon** — LSUIElement=true makes Ozzie menu-bar-only
- **Sandbox disabled** — needed for clipboard access, screenshot capture, and app detection
- **Carbon hotkeys** — most reliable method for global shortcuts on macOS
- **JSON storage** — simple, portable, easy to debug (just open the file)
- **ObservableObject** — StorageManager publishes changes, all views react automatically

2. Project Structure

```
Ozzie/
■■■ project.yml ← XcodeGen config
■■■ Assets.xcassets/ ← App icon assets
■■■ Sources/
■■■ OzzieApp.swift ← @main entry point
■■■ AppDelegate.swift ← Menu bar + popover coordinator
■■■ Info.plist ← App metadata (LSUIElement=true)
■■■ Ozzie.entitlements ← Security permissions
■■■ Models/
■ ■■■ OzzieTask.swift ← Core task model
■ ■■■ SubTask.swift ← Subtask model
■ ■■■ TaskContext.swift ← Context snippet model
■■■ Storage/
■ ■■■ StorageManager.swift ← JSON persistence + CRUD
■■■ Views/
■ ■■■ TaskListView.swift ← Main popover (task list)
■ ■■■ TaskRowView.swift ← Compact task row
■ ■■■ TaskDetailView.swift ← Expanded task editor
■ ■■■ NewTaskView.swift ← Create task sheet
■ ■■■ TaskPickerView.swift ← Context → task attachment
■■■ Services/
■ ■■■ HotkeyManager.swift ← Cmd+Option+N/A/S
■ ■■■ ScreenCaptureService.swift ← Screenshot + OCR
■ ■■■ NotificationManager.swift ← Deadline reminders
■ ■■■ ActiveAppService.swift ← Frontmost app detection
■■■ Resources/
■■■ WizardIcon.swift ← 8-bit pixel art icon
```

3. App Entry & Lifecycle

Sources/OzzieApp.swift

The **@main** entry point. Bootstraps the app with no visible windows.

- Creates `AppDelegate` via `@NSApplicationDelegateAdaptor`
- Returns an empty `Settings` scene — the app is entirely menu-bar-driven
- This is the only file with the `@main` attribute

Sources/AppDelegate.swift

The **central coordinator**. Manages the status bar item, popover, hotkey routing, and notification setup.

Startup sequence (applicationDidFinishLaunching):

- **1.** setupMenuBar() — Creates NSStatusItem with wizard icon
- **2.** setupPopover() — 380×500 NSPopover with TaskListView
- **3.** setupHotkeys() — Registers Cmd+Option+N/A/S via HotkeyManager
- **4.** setupEventMonitor() — Click-outside-to-dismiss listener
- **5.** setupNotifications() — Requests permission, schedules 9AM daily brief
- **6.** setupPopoverResetObserver() — Listens for .resetPopoverContent to swap views back

Hotkey handlers:

- **handleNewTask()** — Gets selected text + source app → creates OzzieTask → opens popover
- **handleAddContext()** — Gets selected text → swaps popover to TaskPickerView
- **handleScreenshot()** — Launches screencapture → OCR → swaps popover to TaskPickerView

Dependency map: Uses StorageManager, HotkeyManager, ActiveAppService, ScreenCaptureService, OzzieNotificationManager, WizardIcon, TaskListView, TaskPickerView

4. Data Models

Models/OzzieTask.swift

The **core task model**. A Codable struct with status, subtasks, contexts, and due dates.

Property	Type	Description
`id`	`UUID`	Unique identifier (auto-generated)
`title`	`String`	Task title (max 100 chars from highlighted text)
`createdAt`	`Date`	Creation timestamp
`dueDate`	`Date?`	Optional due date (date only)
`dueTime`	`Date?`	Optional due time
`status`	`TaskStatus`	todo → inProgress → done (cycles)
`subtasks`	`[SubTask]`	Ordered list of checkable subtasks
`contexts`	`[TaskContext]`	All attached text/screenshot context
`summary`	`String?`	AI-generated summary (future use)
`notificationScheduled`	`Bool`	Whether deadline notifications are set

Computed properties:

- `completedSubtasks` — Count of done subtasks
- `progress` — 0.0 to 1.0 ratio for progress bar
- `isOverdue` — true if past due and not done
- `effectiveDueDate` — Merges dueDate + dueTime into one Date

TaskStatus enum: `.todo` (circle icon) → `.inProgress` (half-circle) → `.done` (checkmark). Cycles on click.

Models/SubTask.swift

A simple **checkable subtask** nested inside OzzieTask.

- `id: UUID` — unique identifier
- `title: String` — subtask text
- `isCompleted: Bool` — checkbox state
- `createdAt: Date` — when it was added

Models/TaskContext.swift

Represents **context attached to a task** — either highlighted text or a screenshot with OCR.

- `content: String` — the text (or OCR result for screenshots)
- `sourceApp: String` — which app the content came from (e.g. 'Safari', 'VS Code')
- `type: ContextType` — `.text` or `.screenshot`
- `screenshotPath: String?` — file path for screenshot images
- `displayIcon` — computed SF Symbol: doc.text or camera.viewfinder

5. Storage Layer

Storage/StorageManager.swift

Singleton **ObservableObject** managing all task persistence. Stores JSON at `~/Library/Application Support/Ozzie/tasks.json`.

Storage paths:

- `~/Library/Application Support/Ozzie/tasks.json` — all task data
- `~/Library/Application Support/Ozzie/screenshots/` — captured images (PNG)

CRUD methods:

Method	What it does
`addTask(_:)`	Appends task, saves, triggers @Published update
`updateTask(_:)`	Finds by ID, replaces, saves
`deleteTask(_:)`	Removes by ID, saves
`addContextToTask(taskId:, context:)`	Appends context to specific task
`addSubtask(taskId:, subtask:)`	Appends subtask to specific task
`toggleSubtask(taskId:, subtaskId:)`	Flips isCompleted on subtask
`saveScreenshot(_:) → String?`	Writes PNG data, returns file path
`searchTasks(query:)`	Searches titles + context content

Query properties:

- `activeTasks` — sorted by creation date, excludes .done
- `completedTasks` — only .done tasks

How it works: Every mutation calls `saveTasks()` which JSON-encodes the entire tasks array to disk. The `@Published var tasks` property triggers SwiftUI view updates automatically via EnvironmentObject.

6. Views (UI Layer)

All views receive `StorageManager` as an `@EnvironmentObject`. The popover content starts as TaskListView and can temporarily swap to TaskPickerView during hotkey flows.

Views/TaskListView.swift

The **main popover content**. Shows a searchable list of active tasks.

Layout (top to bottom):

- **Header bar** — Wizard icon + 'Ozzie' title + New Task button (+)
- **Search bar** — Filters tasks by title (magnifying glass icon)
- **Task list** — ScrollView of TaskRowView items
- **Completed section** — Collapsible, shows done tasks with delete option
- **Empty state** — Shown when no tasks exist (wizard icon + hint text)

Navigation: Tapping a task row sets `selectedTask` and shows TaskDetailView inline (replaces the list).

Context menu: Right-click any task for status change or delete.

Views/TaskRowView.swift

A **compact row** for a single task in the list.

- **Status button** (left) — Cycles todo → inProgress → done on click
- **Title** — Bold, single line, strikethrough if done
- **Progress indicator** — '2/5' subtask count if subtasks exist
- **Context badge** — Number of attached contexts
- **Due date** — Red if overdue, orange for today, gray otherwise

Views/TaskDetailView.swift

The **expanded task editor**. Shown when you click a task.

Sections:

- **Back button** — Returns to task list
- **Title section** — Tap to edit inline
- **Status pills** — Three buttons to set status directly
- **Date section** — Toggle due date on/off, date + time pickers
- **Subtasks** — Progress bar + list of toggleable subtasks + 'Add subtask' field
- **Context section** — Each context shows source app, timestamp, preview (expandable), screenshots inline

Includes nested ContextItemView — handles expand/collapse of context previews and renders screenshots from disk.

Views/NewTaskView.swift

Sheet modal for **creating a task from scratch**.

- Title text field
- Optional due date toggle with date + time pickers
- Inline subtask creation (type + Enter to add, shows list)
- Calls `storageManager.addTask()` and schedules notifications

Views/TaskPickerView.swift

Shown during **Cmd+Option+A** and **Cmd+Option+S** flows. Lets the user pick which task to attach context to.

- **Context preview** — shows the captured text/screenshot and source app
- **Search bar** — fuzzy search through existing tasks
- **'Create new task'** option — creates a task with the context pre-attached
- **Task list** — click any task to attach the context to it
- Posts `.resetPopoverContent` notification when done (AppDelegate swaps back to TaskListView)

7. Services

All services are **singletons** accessed via `.shared`.

Services/HotkeyManager.swift

Registers **global keyboard shortcuts** using the Carbon API.

How it works:

- Installs a Carbon `EventHandler` for `kEventHotKeyPressed`
- Registers three hotkeys with signature 'OZZI' (0x4F5A5A49)
- Each hotkey fires a callback: `onNewTask`, `onAddContext`, `onScreenshot`
- AppDelegate sets these callbacks during `setupHotkeys()`
- Requires **Accessibility permission** in System Settings

Services/ScreenCaptureService.swift

Handles **interactive screenshot capture** and **OCR text recognition**.

Flow:

- `captureSelection()` runs `/usr/sbin/screencapture -i` (interactive selection)
- Saves to a temp file, reads the image data
- `performOCR()` uses Vision framework's `VNRecognizeTextRequest`
- Uses `.accurate` recognition level with language correction enabled
- Returns both the raw image data and extracted text to the callback
- Requires **Screen Recording permission** in System Settings

Services/NotificationManager.swift

Schedules **deadline reminders** and a **daily morning brief**.

Notification types:

Type	When	Content
1-hour warning	60 min before due	'Task X is due in 1 hour'
15-min warning	15 min before due	'Task X is due in 15 minutes!'
At due time	Exact due time	'Task X is due now!'
Daily brief	9:00 AM daily	'You have N active, M due today, K overdue'

- Uses `UNUserNotificationCenter` with calendar-based triggers
- Notification IDs are `ozzie-{taskId}-{1h/15m/due}` for easy removal

Services/ActiveAppService.swift

Detects the **frontmost application** and **grabs selected text**.

- `frontmostAppName()` — returns the active app's localized name via `NSWorkspace`
- `getSelectedText()` — simulates Cmd+C via `CGEvent`, reads from `NSPasteboard`, then restores the previous clipboard content
- Used by AppDelegate in all three hotkey handlers to capture source context
- Requires **Accessibility permission** for CGEvent posting

8. Resources & Configuration

Resources/WizardIcon.swift

Generates the **8-bit pixel art wizard** icon programmatically.

- `pixelMap` — 18×18 grid of UInt8 values defining the wizard shape
- `createMenuBarIcon()` — Black template image (macOS auto-tints for light/dark)
- `createLargeIcon(size:)` — Colored version: purple hat, tan face, dark blue robe, brown staff
- Template images ensure the icon matches the system menu bar style automatically

project.yml

XcodeGen project definition.

- Target: macOS application, deployment target 14.0
- Swift 5.9, automatic code signing
- References Info.plist and Ozzie.entitlements
- Run `xcodegen generate` to create the .xcodeproj

9. Data Flow Diagrams

A. New Task from Highlighted Text (Cmd+Option+N)

1. User highlights text in any app
2. Presses Cmd+Option+N
3. Carbon event handler fires → HotkeyManager.onNewTask callback
4. AppDelegate.handleNewTask() runs:

a. ActiveAppService.frontmostAppName() → gets source app
b. ActiveAppService.getSelectedText() → simulates Cmd+C → reads clipboard
c. Creates OzzieTask with title (first line of text)
d. Creates TaskContext with full text + source app name
e. StorageManager.addTask() → saves to JSON → @Published triggers UI update
f. Shows popover with updated TaskListView

B. Add Context to Existing Task (Cmd+Option+A)

1. User highlights text in any app
2. Presses Cmd+Option+A
3. HotkeyManager.onAddContext → AppDelegate.handleAddContext()
4. Gets selected text + source app
5. Swaps popover content to TaskPickerView (with context text pre-loaded)
6. User searches/selects a task (or creates new)
7. TaskContext created and attached → StorageManager.addContextToTask()
8. TaskPickerView posts .resetPopoverContent → AppDelegate swaps back to TaskListView

C. Screenshot Capture (Cmd+Option+S)

1. User presses Cmd+Option+S
2. HotkeyManager.onScreenshot → AppDelegate.handleScreenshot()
3. 0.3s delay (lets UI settle)
4. ScreenCaptureService.captureSelection() → runs /usr/sbin/screencapture -i
5. User drags to select screen region
6. Image saved to temp file → read as Data
7. Vision framework OCR extracts text from image
8. StorageManager.saveScreenshot() → saves PNG to screenshots/ dir
9. TaskPickerView shown with screenshot preview + OCR text
10. User picks task → TaskContext (type: .screenshot) attached

D. Notification Flow

1. User sets due date on task (via NewTaskView or TaskDetailView)
2. View calls OzzieNotificationManager.scheduleDeadlineNotifications(for: task)
3. Three UNNotificationRequests created:

- ozzie-{id}-1h → fires 60min before due
- ozzie-{id}-15m → fires 15min before due
- ozzie-{id}-due → fires at due time

4. On app launch, scheduleDailyBrief() sets recurring 9AM notification
5. When task deleted, removeNotifications(for:) cleans up pending requests

10. Permissions & Entitlements

Permission	Why Needed	Where Configured
Accessibility	Global hotkeys + CGEvent for clipboard	System Settings > Privacy
Screen Recording	screencapture -i for screenshots	System Settings > Privacy
Notifications	Deadline reminders + daily brief	Prompted on first launch
App Sandbox: OFF	Clipboard, automation, file access	Ozzie.entitlements
Automation (AppleEvents)	Detect frontmost app name	Ozzie.entitlements + Info.plist
File Access (user-selected)	Read/write screenshots + JSON	Ozzie.entitlements

11. Hotkey Reference

Shortcut	Action	Handler	What Happens
`Cmd+Option+N`	New Task	`handleNewTask()`	Captures text → creates task → opens popover
`Cmd+Option+A`	Add Context	`handleAddContext()`	Captures text → opens task picker
`Cmd+Option+S`	Screenshot	`handleScreenshot()`	Region select → OCR → opens task picker

Carbon key codes used: kVK_ANSI_N (0x2D), kVK_ANSI_A (0x00), kVK_ANSI_S (0x01). Modifiers: cmdKey | optionKey.